

Итераторы

1. Что это?

Итераторы – это типы данных, позволяющие перебирать элементы (чаще всего, контейнеров).

С итераторами явно вы сталкивались в ЛР5, когда знакомились с алгоритмами STL, а неявно сталкиваетесь каждый раз, когда используете range-based for для контейнеров. Думаю, никто не будет спорить с тем, что такой формат работы удобен, а возможность применять стандартные алгоритмы сильно упрощает работу. Но что делать, если вы используете свой тип для хранения набора данных, у которого итераторов нет? Определить свои конечно же!

Что «под капотом» у итератора? В самом простом случае итератор является обёрткой над указателем. Тем не менее, он может быть и более сложным типом, главное чтобы обеспечивал требуемый функционал.

В C++20 итераторы поменяли свой вид, теперь они основываются на [концепциях](#). Современному C++ посвящён 3 семестр, так что пока знакомимся с «классическими» итераторами.

2. Типы итераторов

Итераторы с т.з. доступа на чтение можно расположить по иерархии (таблица 1). Каждый следующий итератор в иерархии добавляет к предыдущему новые операции или новые требования к уже имеющимся операциям.

Особняком стоит `Output iterator`, который отвечает за изменение данных. Любой из итераторов на чтение может одновременно обладать свойствами `Output iterator`, тем самым поддерживая изменение данных. Такие итераторы называются **изменяемыми**. Итераторы, которые не являются `Output iterator`, называются **неизменяемыми**.

Таблица 1 – Операции, которые должен поддерживать итератор.

Итератор	Операции
1. Input iterator – обеспечивает один проход в одну сторону	1) == 2) != 3) -> 4) * 5) ++ (правосторонний) 6) ++ (левосторонний)
2. Forward iterator – обеспечивает многопроходный однонаправленный доступ	Не добавляет новых операций
3. Bidirectional iterator – обеспечивает множественные проходы в любом направлении	7) -- (правосторонний) 8) -- (левосторонний)
4. Random access iterator – обеспечивает доступ к любому элементу за постоянное время	9) [] 10) + (сдвиг + итератор) 11) + (итератор + сдвиг) 12) - (итератор – сдвиг) 13) - (итератор – итератор) 14) += 15) -= 16) > 17) < 18) >= 19) <=
5. Contiguous iterator – обеспечивает доступ к любому элементу за постоянное время	Не добавляет новых операций
Output iterator – обеспечивает модификацию данных	Не добавляет новых операций

3. Как написать свой итератор

В контейнерах мы чаще всего имеем дело с Random access iterator, который также является Output iterator. Исходя из таблицы выше, для него должно быть определено 19 операторов. Рассмотрим пример такого итератора для строки-обёртки над `char*`. Объявление этого класса приведено в примере 1, уже добавлены функции получения начального и конечного итераторов.

Пример 1. Класс String

```
1  class String
2  {
3      char* _symbols;
4      size_t _len; // Фактическая длина строки
5      size_t _capacity; // Максимальная длина строки
6  public:
7      String(const char* str);
8      size_t length() const
9      {
10         return _len;
11     }
12     size_t size() const
13     {
14         return _len;
15     }
16     size_t capacity() const
17     {
18         return _capacity - 1;
19     }
20     ~String()
21     {
22         delete[] _symbols;
23     }
24     String(String const& other);
25     String& operator=(String const& other);
26     StringIterator begin() const;
27     StringIterator end() const;
28 };
```

Строка 20 из примера 1 обусловлена тем, что `end()` (строка 31) должна возвращать итератор на элемент, стоящий после последнего (рис. 1), и область памяти, на которую он ссылается, должна быть «легальной». Таким образом, принимая, что была выделена область памяти размером `_capacity`, пользователю сообщаем о том, что максимальное кол-во элементов, которое может поместиться (*capacity* – ёмкость), – `_capacity - 1`. Также возможен подход, когда хранится и возвращается пользователю именно значение `_capacity`, но тогда нужно выделять память под массив размера `_capacity + 1`.

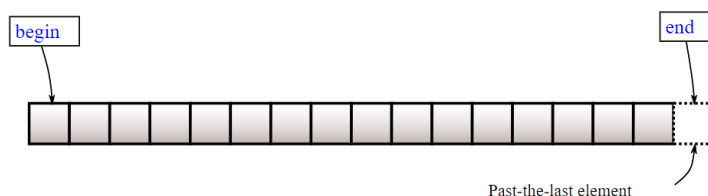


Рисунок 1 – Фантомный элемент в хранилище элементов

Итак, у итератора должна быть категория и `using` для 4 типов: тип значения, тип расстояния (смещения), тип указателя и тип ссылки. В интернете вы можете наткнуться на рекомендации делать свои итераторы наследниками стандартного итератора `std::iterator`. В настоящий момент этот подход является устаревшим. Его результат эквивалентен «ручному» определению этих `using`'ов. Строки 5-9 примера 2 реализуют `using`'и.

Строка 11 – конструктор с параметрами. Поскольку класс не будет работать с ресурсами, в соответствии с правилом нуля, ему не нужно определять конструктор копирования, оператор присваивания копированием и деструктор. Далее объявлены операторы, требуемые `Random access iterator`. Рассмотрим их реализации.

Итераторы равны тогда, когда они указывают на один и тот же элемент. Таким образом, операторы сравнения сравнивают значения лежащих «под капотом» указателей. Оператор разыменования должен возвращать объект. Т.к. итератор является `Output iterator`, то возвращаем объект по ссылке. Объект получаем разыменованием указателя. Оператор `->` должен предоставлять доступ к полям элемента при обращении по указателю. Так что возвращаем указатель.

Пример 2. Объявление класса `StringIterator`

```
1  class StringIterator
2  {
3      char* _pos;
4  public:
5      using iterator_category = std::random_access_iterator_tag;
6      using value_type = char; // Тип значения
7      using difference_type = int; // Тип расстояния
8      using pointer = char*; // Тип указателя
9      using reference = char&; // Тип ссылки
10
11     StringIterator(char* pos) : _pos(pos) {}
12
13     bool operator==(StringIterator const& other) const // 1
14     {
15         return _pos == other._pos;
16     }
17
18     bool operator!=(StringIterator const& other) const // 2
19     {
20         return _pos != other._pos;
21     }
22
23     reference operator*() const // 3
24     {
25         return *_pos;
26     }
27
28     pointer operator->() const // 4
29     {
30         return _pos;
31     }
32
```

Далее определяем операторы инкремента и декремента. Как вы помните, они существуют в двух вариантах (префиксном и постфиксном). Для того, чтобы компьютер отличал префиксную версию от постфиксной, вводится фиктивный параметр типа `int`. Он не будет использоваться, поэтому ему не нужно даже имя. Префиксный вариант меняет значение указателя, а затем возвращает итератор с модифицированным значением. Постфиксный вариант модифицирует указатель, но возвращает итератор до модификации.

Квадратные скобки позволяют получить элемент, стоящий на позиции `index` относительно текущей (на которую указывает `_pos`). Этот механизм аналогичен доступу к `index`'ому элементу массива, начинающегося с ячейки памяти, на которую указывает `_pos`. Оператор сложения позволяет получить новый итератор, указывающий на позицию, отстоящую от текущей на `diff`. Сложение с итератором слева может быть объявлено как внутри класса, так и вне его. Сложение с итератором справа (слева `diff`) должно быть внешней функцией. Его можно реализовать с помощью геттера для `_pos` или сделать дружественной функцией.

Продолжение примера 2

```
33     StringIterator& operator++() // 5
34     {
35         ++_pos;
36         return *this;
37     }
38
39     StringIterator operator++(int) // 6
40     {
41         StringIterator tmp = *this;
42         ++_pos;
43         return tmp;
44     }
45
46     StringIterator& operator--() // 7
47     {
48         --_pos;
49         return *this;
50     }
51
52     StringIterator operator--(int) // 8
53     {
54         StringIterator tmp = *this;
55         --_pos;
56         return tmp;
57     }
58
59     reference operator[](size_t index) const // 9
60     {
61         return _pos[index];
62     }
63
64     StringIterator operator+(difference_type const& diff) const // 10
65     {
66         return StringIterator(_pos + diff);
67     }
68
```

Аналогичным образом определяется вычитание смещения из итератора. А вот вычитание итератора из смещения смысла не имеет. Второй оператор вычитания используется для нахождения расстояния между итераторами. Для этого используется функция `std::distance` из стандартной библиотеки C++. Поскольку и правый, и левый операнды являются итераторами, он может быть объявлен как внутри класса, так и вне. Операторы сравнения тривиальны.

Продолжение примера 2

```
69         friend inline StringIterator operator+(difference_type const& diff, // 11
70                                             StringIterator const& it);
71
72         StringIterator operator-(difference_type const& diff) const      // 12
73         {
74             return StringIterator(_pos - diff);
75         }
76
77         difference_type operator-(StringIterator const& other) const    // 13
78         {
79             return std::distance(other._pos, _pos);
80         }
81
82         StringIterator& operator+=(difference_type const& diff) // 14
83         {
84             _pos += diff;
85             return *this;
86         }
87
88         StringIterator& operator-=(difference_type const& diff); // 15
89         {
90             _pos -= diff;
91             return *this;
92         }
93
94         bool operator>(StringIterator const& other) const // 16
95         {
96             return _pos > other._pos;
97         }
98
99         bool operator<(StringIterator const& other) const // 17
100        {
101            return _pos < other._pos;
102        }
103
104        bool operator>=(StringIterator const& other) const // 18
105        {
106            return _pos >= other._pos;
107        }
108
109        bool operator<=(StringIterator const& other) const // 19
110        {
111            return _pos <= other._pos;
112        }
113    };
114
```

Продолжение примера 2

```
115 StringIterator operator+(StringIterator::difference_type const& diff,  
116     StringIterator const& it)  
117 {  
118     return StringIterator(it._pos + diff);  
119 }
```

Теперь осталось «привязать» итератор к классу-строке, определив методы `begin` и `end` (пример 3).

Пример 3. Методы String для работы с итераторами.

```
1 StringIterator String::begin() const  
2 {  
3     return StringIterator(&_amp;symbols[0]);  
4 }  
5  
6 StringIterator String::end() const  
7 {  
8     return StringIterator(&_amp;symbols[_len]);  
9 }
```

На рисунке 2 приведён пример работы класса с алгоритмами STL (`for_each`, `find`, `count_if`, `sort`, `swap`, `max_element`, `min_element`), а также операторов разыменования, `[]` и арифметических операций с итераторами. Код приведён в примере 4.

```
String: ABCDEFG 12345 asdfgh  
Symbols: A|B|C|D|E|F|G| |1|2|3|4|5| |a|s|d|f|g|h|  
First pos of 's' is 15  
Numbers: 5  
Upper case to lower: abcdefg 12345 asdfgh  
7 symbols were replaced  
Sorted: 12345aabcddeffgghs  
Incremented: !!23456bbcdeefgghhit  
New first: A!23456bbcdeefgghhit  
Symbol by []: 5  
Changed symbol: A!234P6bbcdeefgghhit  
Reversed (except first & last symbols): Aihhggfeedcbb6P432!t  
Max value: t  
Min value: !  
Swap min & max: Aihhggfeedcbb6P432t!
```

Рисунок 2 – Результат работы класса со стандартными алгоритмами

Пример 4. Взаимодействие класса со стандартными алгоритмами.

```
1  #include "String.h"
2  #include <iostream>
3  #include <algorithm>
4
5  int main()
6  {
7      String str("ABCDEFGH 12345 asdfgh");
8      std::cout << "String: ";
9      std::for_each(str.begin(), str.end(), [](auto i) {std::cout << i; });
10     std::cout << std::endl << "Symbols: ";
11     for (auto i : str)
12         std::cout << i << "|";
13     std::cout << std::endl << "First pos of \'s\' is ";
14     std::cout << std::find(str.begin(), str.end(), 's') - str.begin() << std::endl;
15     std::cout << "Numbers: " << std::count_if(str.begin(), str.end(),
16         [](auto i) {return (i >= '0') && (i <= '9'); }) << std::endl;
17     std::cout << "Upper case to lower: ";
18     std::cout << std::endl << std::count_if(str.begin(), str.end(),
19         [](auto& i) mutable {if ((i >= 'A') && (i <= 'Z')) { i = 'a' + i - 'A';
20             std::cout << i; return true; } std::cout << i; return false; });
21     std::cout << " symbols were replaced" << std::endl;
22     std::sort(str.begin(), str.end());
23     std::cout << "Sorted: ";
24     std::for_each(str.begin(), str.end(), [](auto i) {std::cout << i; });
25     for (auto &i : str)
26         i = i + 1;
27     std::cout << std::endl << "Incremented: ";
28     for (auto i : str)
29         std::cout << i;
30     *str.begin() = 'A';
31     std::cout << std::endl << "New first: ";
32     for (auto i : str)
33         std::cout << i;
34     std::cout << std::endl << "Symbol by []: " << (str.begin() + 2)[3];
35     (str.begin() + 2)[3] = 'P';
36     std::cout << std::endl << "Changed symbol: ";
37     for (auto i : str)
38         std::cout << i;
39     std::cout << std::endl << "Reversed (except first & last symbols): ";
40     std::reverse(str.begin() + 1, str.end() - 1);
41     for (auto i : str)
42         std::cout << i;
43     std::cout << std::endl << "Max value: ";
44     std::cout << *std::max_element(str.begin(), str.end());
45     std::cout << std::endl << "Min value: ";
46     std::cout << *std::min_element(str.begin(), str.end());
47     std::cout << std::endl << "Swap min & max: ";
48     std::swap(*std::max_element(str.begin(), str.end()),
49         *std::min_element(str.begin(), str.end()));
50     for (auto i : str)
51         std::cout << i;
52     return 0;
53 }
```

Приложение 1 – Содержимое файла String.h

```
#pragma once
#include <algorithm>
#include <iterator>

class StringIterator
{
    char* _pos;
public:
    using iterator_category = std::random_access_iterator_tag;
    using value_type = char; // Тип значения
    using difference_type = int; // Тип расстояния
    using pointer = char*; // Тип указателя
    using reference = char&; // Тип ссылки

    StringIterator(char* pos) : _pos(pos) {}

    bool operator==(StringIterator const& other) const // 1
    {
        return _pos == other._pos;
    }

    bool operator!=(StringIterator const& other) const // 2
    {
        return _pos != other._pos;
    }

    reference operator*() const // 3
    {
        return *_pos;
    }

    pointer operator->() const // 4
    {
        return _pos;
    }

    StringIterator& operator++() // 5
    {
        ++_pos;
        return *this;
    }

    StringIterator operator++(int) // 6
    {
        StringIterator tmp = *this;
        ++_pos;
        return tmp;
    }

    StringIterator& operator--() // 7
    {
        --_pos;
        return *this;
    }
}
```

```

StringIterator operator--(int) // 8
{
    StringIterator tmp = *this;
    --_pos;
    return tmp;
}

reference operator[](size_t index) const // 9
{
    return _pos[index];
}

StringIterator operator+(difference_type const& diff) const // 10
{
    return StringIterator(_pos + diff);
}

friend inline StringIterator operator+(difference_type const& diff, // 11
    StringIterator const& it);

StringIterator operator-(difference_type const& diff) const // 12
{
    return StringIterator(_pos - diff);
}

difference_type operator-(StringIterator const& other) const // 13
{
    return std::distance(other._pos, _pos);
}

StringIterator& operator+=(difference_type const& diff) // 14
{
    _pos += diff;
    return *this;
}

StringIterator& operator-=(difference_type const& diff) // 15
{
    _pos -= diff;
    return *this;
}

bool operator>(StringIterator const& other) const // 16
{
    return _pos > other._pos;
}

bool operator<(StringIterator const& other) const // 17
{
    return _pos < other._pos;
}

bool operator>=(StringIterator const& other) const // 18
{
    return _pos >= other._pos;
}

```

```

    }

    bool operator<=(StringIterator const& other) const // 19
    {
        return _pos <= other._pos;
    }
};

class String
{
    char* _symbols;
    size_t _len;
    size_t _capacity;
public:
    String(const char* str);
    size_t length() const
    {
        return _len;
    }
    size_t size() const
    {
        return _len;
    }
    size_t capacity() const
    {
        return _capacity - 1;
    }
    ~String()
    {
        delete[] _symbols;
    }
    String(String const& other);
    String& operator=(String const& other);
    StringIterator begin() const;
    StringIterator end() const;
};

```

Приложение 2 – Содержимое файла String.cpp

```
#include "String.h"

String::String(const char* str)
{
    if ((str == nullptr) || (str[0] == '\\0'))
    {
        _len = 0;
        _capacity = 1;
        _symbols = new char[1];
    }
    else
    {
        size_t length = 0;
        while (str[length] != '\\0')
            ++length;
        _symbols = new char[length + 1];
        _len = length;
        _capacity = length + 1;
        for (size_t i = 0; i < _len; ++i)
            _symbols[i] = str[i];
    }
}

String::String(String const& other) : _len(other._len), _capacity(other._capacity)
{
    if (_capacity == 1)
        _symbols = new char[1];
    else
    {
        _symbols = new char[_capacity];
        for (size_t i = 0; i < _len; ++i)
            _symbols[i] = other._symbols[i];
    }
}

String& String::operator=(String const& other)
{
    if (this != &other)
    {
        if (_capacity != other._capacity)
        {
            delete[] _symbols;
            _capacity = other._capacity;
            _symbols = new char[_capacity];
        }
        _len = other._len;
        for (size_t i = 0; i < _len; ++i)
            _symbols[i] = other._symbols[i];
    }
    return *this;
}
```

```

StringIterator operator+(StringIterator::difference_type const& diff,
    StringIterator const& it)
{
    return StringIterator(it._pos + diff);
}

StringIterator String::begin() const // Можно было определить в заголовке
{
    return StringIterator(&_amp;_symbols[0]);
}

StringIterator String::end() const // Можно было определить в заголовке
{
    return StringIterator(&_amp;_symbols[_len]);
}

```

Приложение 3 – Содержимое файла Source.cpp

```
#include "String.h"
#include <iostream>
#include <algorithm>

int main()
{
    String str("ABCDEFGH 12345 asdfgh");
    std::cout << "String: ";
    std::for_each(str.begin(), str.end(), [](auto i) {std::cout << i; });
    std::cout << std::endl << "Symbols: ";
    for (auto i : str)
        std::cout << i << "|";
    std::cout << std::endl << "First pos of \'s\' is ";
    std::cout << std::find(str.begin(), str.end(), 's') - str.begin() << std::endl;
    std::cout << "Numbers: " << std::count_if(str.begin(), str.end(),
        [](auto i) {return (i >= '0') && (i <= '9'); }) << std::endl;
    std::cout << "Upper case to lower: ";
    std::cout << std::endl << std::count_if(str.begin(), str.end(),
        [](auto& i) mutable {if ((i >= 'A') && (i <= 'Z')) {
            i = 'a' + i - 'A';
            std::cout << i; return true;
        } std::cout << i; return false; });
    std::cout << " symbols were replaced" << std::endl;
    std::sort(str.begin(), str.end());
    std::cout << "Sorted: ";
    std::for_each(str.begin(), str.end(), [](auto i) {std::cout << i; });
    for (auto &i : str)
        i = i + 1;
    std::cout << std::endl << "Incremented: ";
    for (auto i : str)
        std::cout << i;
    *str.begin() = 'A';
    std::cout << std::endl << "New first: ";
    for (auto i : str)
        std::cout << i;
    std::cout << std::endl << "Symbol by []: " << (str.begin() + 2)[3];
    (str.begin() + 2)[3] = 'P';
    std::cout << std::endl << "Changed symbol: ";
    for (auto i : str)
        std::cout << i;
    std::cout << std::endl << "Reversed (except first & last symbols): ";
    std::reverse(str.begin() + 1, str.end() - 1);
    for (auto i : str)
        std::cout << i;
    std::cout << std::endl << "Max value: ";
    std::cout << *std::max_element(str.begin(), str.end());
    std::cout << std::endl << "Min value: ";
    std::cout << *std::min_element(str.begin(), str.end());
    std::cout << std::endl << "Swap min & max: ";
    std::swap(*std::max_element(str.begin(), str.end()),
        *std::min_element(str.begin(), str.end()));
    for (auto i : str)
        std::cout << i;
    return 0;
}
```

}